

Wine Quality Prediction: An End-to-End Statistical Analysis¹

Tena Alshaikhli, Roopleen Kaur, and Timothy Hays

University of San Diego

AAI-500: Probability and Statistics for Artificial Intelligence

Professor Leon Shpaner

Group 1 | Final Team Project

<https://github.com/Denaalshaikhli/Wine-Quality-Prediction>

June 22, 2026

¹Artificial intelligence tools were used to support code refinement and notebook formatting. Such tools were also used to assist with manuscript formatting and to help verify adherence to APA citation guidelines.

Abstract

This study investigates whether physicochemical properties of wine, obtained through standard laboratory testing, can reliably predict wine quality classifications. Using the UCI Wine Quality Dataset (Cortez et al., 2009), which combines 5,320 cleaned samples of red and white Vinho Verde wine from Portugal, the analysis applies an end-to-end statistical pipeline across seven analytical modules. The analysis includes descriptive statistics, sampling distribution verification through the Central Limit Theorem, confidence interval estimation, Bayesian Network probabilistic modeling, logistic regression via a Generalized Linear Model (GLM), and machine learning classification using Logistic Regression, Naive Bayes, Random Forest, and Gradient Boosting. Results demonstrate that alcohol content ($r = +0.47$) is the strongest positive linear predictor of wine quality, while density ($r = -0.33$) shows the strongest negative linear relationship and volatile acidity remains a substantial negative quality signal. The Bayesian Network shows that combining high alcohol with low volatile acidity raises the posterior probability of good quality classification to approximately 89 percent, compared to a prior of 62.6 percent. The GLM achieves an AUC-ROC of 0.83 on the held-out test set, while Random Forest achieves the highest classification accuracy at 78.3 percent among all models evaluated. These findings support the feasibility of a data-driven quality prediction system that can supplement or partially replace costly human tasting panels in wine production.

Introduction

Wine quality assessment has traditionally depended on sensory evaluation by trained human experts. This process is inherently subjective, time-consuming, and costly. For producers of Vinho Verde, a Portuguese wine variety, the absence of an objective and scalable quality assessment method creates operational challenges at every stage of production. This project proposes a data-driven solution grounded in the statistical methods taught throughout AAI-500.

Can physicochemical properties of wine, measurable through standard laboratory testing, reliably predict whether a wine will be rated as good or poor quality by human tasters?

The dataset used in this analysis is the UCI Wine Quality Dataset (Cortez et al., 2009), one of the most widely cited datasets in machine learning research. It combines red and white Vinho Verde wine samples from the north of Portugal, providing a total of 6,497 original observations with 11 physicochemical input variables and one sensory quality score. After data cleaning, the working dataset contains 5,320 samples composed of 1,359 red wine samples and 3,961 white wine samples. Each wine was evaluated by at least three sensory experts on a scale from 0 (very bad) to 10 (excellent), with the median score recorded as the quality rating.

This analysis follows the statistical framework developed across all seven modules of AAI-500. The pipeline begins with descriptive statistics and data visualization (Module 1), proceeds through probability distributions and Bayesian reasoning (Modules 2 and 3), moves to confidence intervals and hypothesis testing (Module 4), Bayesian Networks (Module 5), and Generalized Linear Models (Module 6), concluding with classification and model evaluation (Module 7).

Dataset Overview

All 11 input variables are continuous physicochemical measurements obtained through standard laboratory testing. The target variable is the wine quality score, binarized for classification: wines with a score of 6 or higher are classified as Good Quality (1) and wines with a score below 6 are classified as Poor Quality (0). This threshold follows the convention established in the original

literature (Cortez et al., 2009, p. 549).

Table 1

Data Dictionary for the UCI Wine Quality Dataset

Variable	Description	Unit	Type
fixed acidity	Non-volatile acids	g/dm3	Continuous
volatile acidity	Acetic acid; vinegar smell at high levels	g/dm3	Continuous
citric acid	Adds freshness and flavor	g/dm3	Continuous
residual sugar	Sugar remaining after fermentation	g/dm3	Continuous
chlorides	Salt content	g/dm3	Continuous
free sulfur dioxide	Prevents microbial growth	mg/dm3	Continuous
total sulfur dioxide	Total SO2 (free and bound)	mg/dm3	Continuous
density	Mass per volume	g/cm3	Continuous
pH	Acidity level on a 0 to 14 scale	—	Continuous
sulphates	Antimicrobial and antioxidant additive	g/dm3	Continuous
alcohol	Alcohol percentage by volume	%	Continuous
quality	Sensory score 0 to 10 — TARGET	—	Discrete

Note. Adapted from Cortez et al. (2009). All variables are quantitative measurements from laboratory testing.

Target Variable Distribution

Of the 5,320 samples in the cleaned dataset, 3,332 samples (62.6%) are classified as Good Quality and 1,988 samples (37.4%) are classified as Poor Quality. This moderate class imbalance is addressed through stratified train-test splitting during model evaluation.

Data Cleaning and Preparation

The raw UCI Wine Quality data arrived as two structurally identical tables: 1,599 red wines and 4,898 white wines, each measured on the same 11 physicochemical variables with a sensory quality score as the target outcome (Cortez et al., 2009). The first cleaning pass used standard inspection tools such as `info()`, `describe()`, and `value_counts()` to confirm data types, check plausible ranges, and review the distribution of the target variable before any modeling decisions were made.

Initial Quality Checks

No missing values were present in either source dataset, which meant the main cleaning issues were duplicate observations and the need to preserve wine type after merging. This is an important result because it allowed the project to keep the full set of measured chemical predictors instead of introducing imputation error or discarding large portions of the data.

Duplicate Removal and Dataset Integration

Duplicate rows were removed before exploratory analysis and modeling so repeated chemical profiles would not overweight particular wines. This step eliminated 240 duplicated red-wine rows and 937 duplicated white-wine rows, leaving 1,359 unique red wines and 3,961 unique white wines. A binary `wine_type` indicator was then added, coded as 0 for red wine and 1 for white wine, so the two datasets could be concatenated without losing the distinction between production styles.

Combining the red and white tables served two practical purposes. First, it increased the effective sample size for downstream estimation, which improves statistical stability and reduces the risk that model comparisons are driven by a narrow subset of observations. Second, it broadened the range of chemical profiles available to the models while still allowing wine color to remain an explicit predictor. After concatenation, the rows were shuffled using a fixed seed (`random_state = 42`) to remove ordering artifacts before the train-test split.

Comparing Red and White Wine Profiles

The cleaned combined dataset therefore contains 5,320 wines in total, with white wines forming the majority class. Even before formal modeling, the summary statistics show that the two wine types differ in meaningful chemical ways while still sharing a comparable quality scale.

Table 2

Selected Mean Differences Between Red and White Wines After Cleaning

Variable	Red Wine Mean	White Wine Mean
fixed acidity	8.31	6.84
volatile acidity	0.53	0.28
residual sugar	2.52	5.92
chlorides	0.088	0.046
free sulfur dioxide	15.89	34.89
total sulfur dioxide	46.83	137.19
alcohol	10.43	10.59

Note. Values are recalculated from the deduplicated red- and white-wine subsets used in the final notebook.

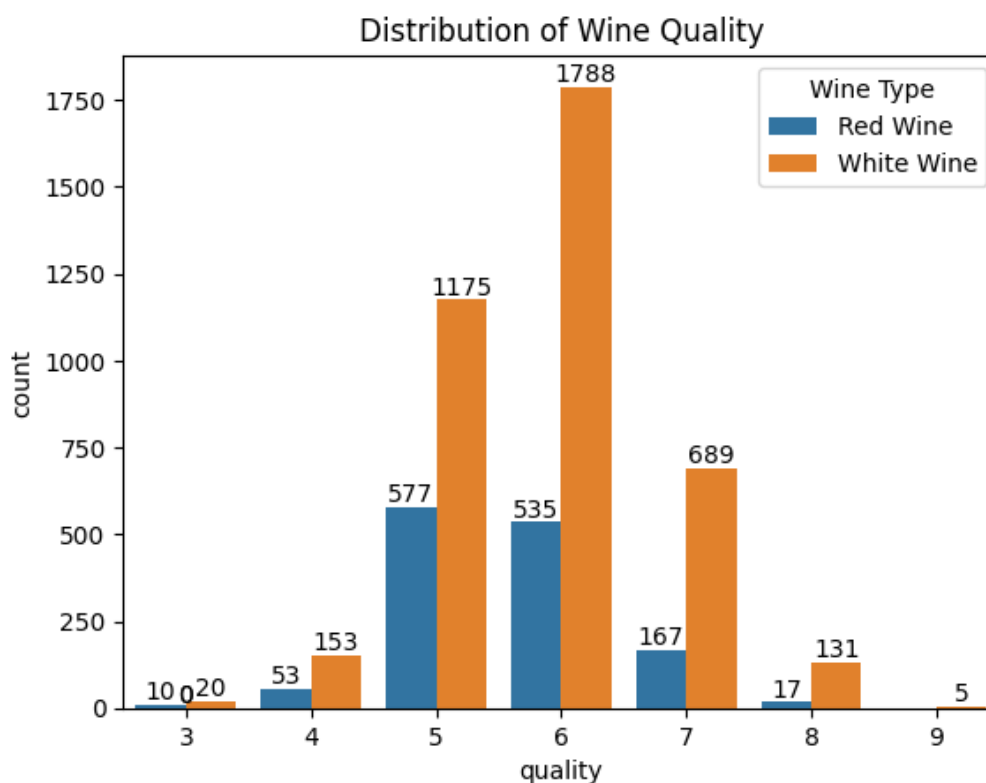
The table above shows that red wines tend to be more acidic and have higher chloride levels, whereas white wines contain substantially more residual sugar and sulfur dioxide. Alcohol content is remarkably similar across the two wine types, which helps explain why the datasets can be studied together without collapsing into two unrelated prediction problems. The cleaned data also preserved the original mid-range quality concentration, with most observations falling between quality scores 5 and 7. Overall, the preparation stage produced a large, internally consistent dataset that was ready for exploratory analysis and model development.

Exploratory Data Analysis

This section uses descriptive and graphical exploration to identify broad structure in the cleaned wine dataset. The purpose here is exploratory rather than confirmatory: to locate imbalance, outliers, skewness, and correlation patterns that later sections interpret more formally through hypothesis testing and predictive modeling.

Dataset Composition and Quality Distribution

The cleaned dataset contains 1,359 red wines and 3,961 white wines, so white wine observations form the majority of the sample. Across both wine types, quality scores range from 3 to 9, but the distribution is concentrated in the middle categories. Scores of 5 and 6 dominate the dataset, score 7 is still common, and the most extreme values at the tails occur relatively rarely.

Figure 1*Wine Quality Distribution by Wine Type*

Note. White wines outnumber red wines in every major quality category, but both wine types are centered in the same mid-quality range.

The count plot above shows that both red and white wines occupy the same general quality band even though the sample sizes differ substantially. This matters because it suggests that wine type alone does not determine quality. Instead, wine type likely interacts with the underlying chemical predictors, which motivates the later use of multivariable models rather than a simple rule based only on color.

Outlier Assessment

Exploratory screening identified substantial outlier activity in several chemical variables, especially those related to acidity, salt content, and sulfur dioxide. These observations were retained rather than removed because they remain chemically plausible and are part of the real variation present in production-scale wine data.

Table 3

Outlier Counts Identified During Exploratory Screening

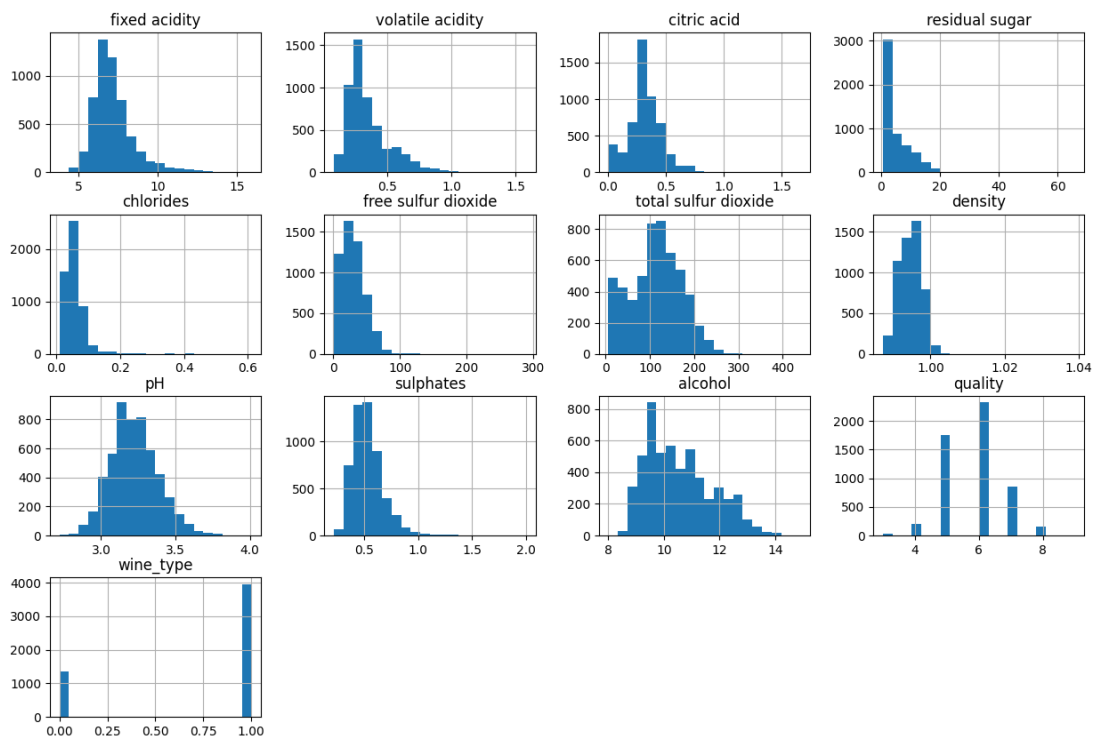
Feature	Outlier Count
fixed acidity	304
volatile acidity	279
chlorides	237
quality	183
sulphates	163
citric acid	143
residual sugar	141
pH	49
free sulfur dioxide	44
total sulfur dioxide	10
density	3
alcohol	1
wine_type	0

Note. Outlier counts are taken from the final notebook's IQR-based screening output.

The outlier pattern is informative in its own right. Fixed acidity, volatile acidity, chlorides, sulphates, and residual sugar all show sizable tails, which means any model that assumes near-normal predictors should be interpreted cautiously. By contrast, density and alcohol are comparatively stable, indicating that not every chemical feature is equally variable or equally difficult to model.

Distributional Shape and Correlation Structure

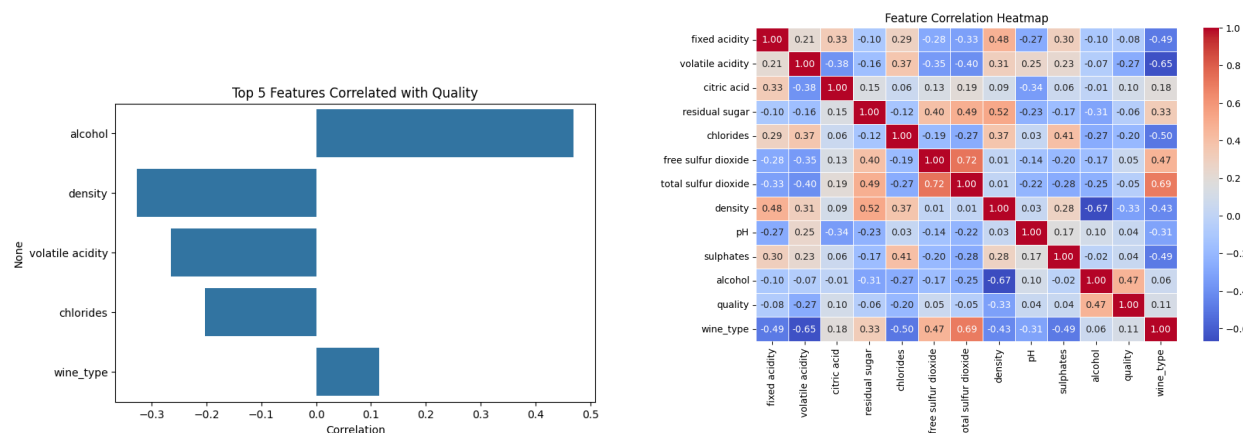
The feature distributions are not uniformly normal. Residual sugar, chlorides, free sulfur dioxide, and total sulfur dioxide are strongly right-skewed, while volatile acidity, citric acid, sulphates, and alcohol show more moderate positive skew. Density is tightly clustered and close to symmetric, and pH is approximately bell-shaped around the low-3 range. These distributional differences suggest that scaling or transformation may help some models, especially when the predictors have long tails or unequal spread.



Additional EDA visual. Distributional histograms from the Word draft show that many physicochemical variables are right-skewed, while pH and density remain comparatively stable. This visual is now preserved in the LaTeX version as part of the exploratory analysis record.

The exploratory correlation scan also revealed a mix of weak and moderate relationships rather than a single dominant linear driver of quality. Alcohol showed the strongest positive association with quality ($r \approx 0.47$), while density showed a moderate negative association ($r \approx -0.33$).

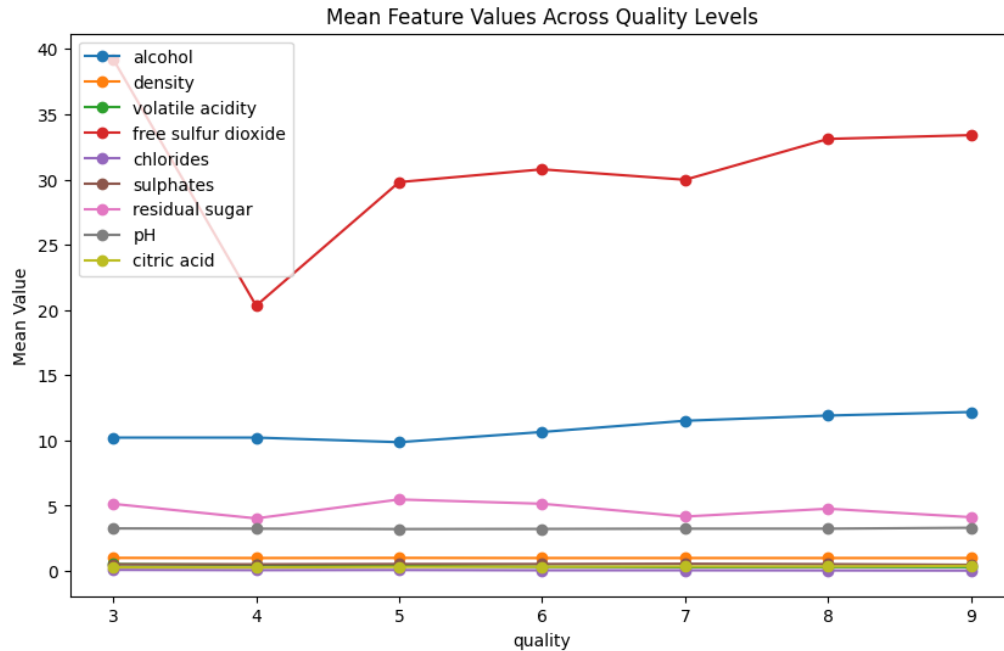
Among the predictors themselves, free sulfur dioxide and total sulfur dioxide were strongly positively correlated ($r \approx 0.72$), and alcohol and density were strongly negatively correlated ($r \approx -0.67$). Those inter-feature relationships point to multicollinearity, which helps explain why some later models gain accuracy by using non-linear interactions rather than relying on a single linear effect per variable.



Additional EDA visuals. The left panel highlights the strongest single-feature correlations with quality, with alcohol emerging as the clearest positive signal. The right panel preserves the full feature-correlation heatmap from the Word draft, showing the strong dependencies among sulfur dioxide, alcohol, density, and wine type that later affect linear-model interpretation.

Grouped Means by Quality Level

Grouped mean comparisons across the quality scores showed one especially consistent pattern: average alcohol content rises steadily as wine quality increases. Other variables such as volatile acidity, density, citric acid, and pH remain flatter or less monotonic across adjacent quality bands, while residual sugar and free sulfur dioxide move in more irregular ways. The practical takeaway from the EDA is therefore not that one variable alone determines quality, but that alcohol supplies the clearest directional signal and should remain central in the later inferential and predictive sections.



Additional EDA visual. Mean feature values across wine-quality levels show the clearest monotonic rise for alcohol, while most other variables move more weakly or nonlinearly across adjacent score bands.

Multicollinearity and Feature Scaling

The Variance Inflation Factor (VIF) analysis revealed varying levels of multicollinearity among the predictor variables. Density, pH, alcohol, fixed acidity, sulphates, total sulfur dioxide, wine type, and volatile acidity all exhibited especially high VIF values, while citric acid, free sulfur dioxide, and chlorides showed more moderate dependence. Residual sugar was the most independent of the major predictors. This pattern is chemically plausible: many of the measurements reflect related fermentation and preservation processes, so the predictors should not be expected to move independently.

	Feature	VIF
7	density	1158.115584
8	pH	661.420676
10	alcohol	110.235933
0	fixed acidity	69.709584
9	sulphates	20.177910
6	total sulfur dioxide	19.941166
11	wine_type	19.485059
1	volatile acidity	11.151995
2	citric acid	9.438095
5	free sulfur dioxide	8.532951
4	chlorides	5.522006
3	residual sugar	3.320395

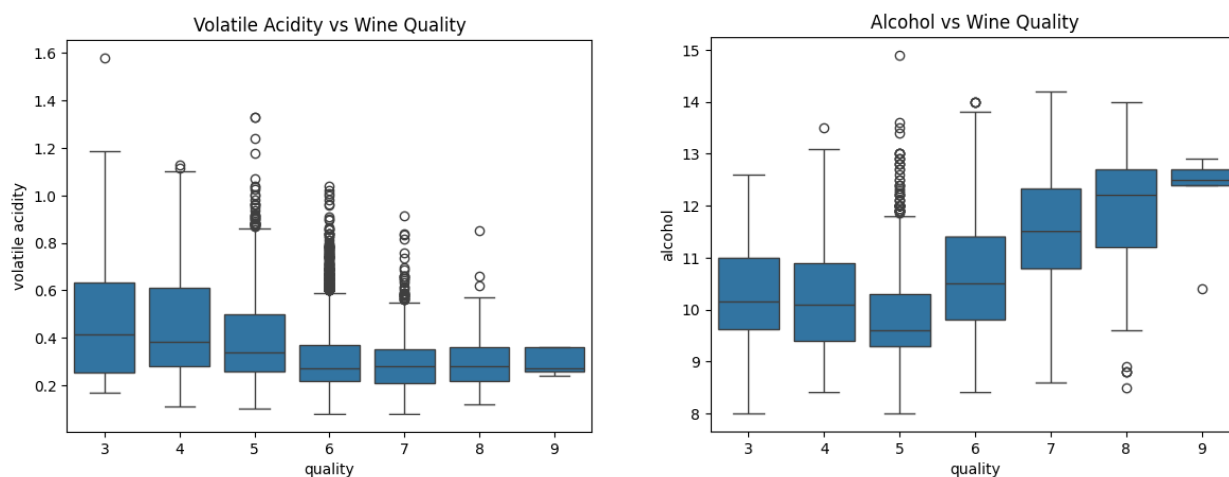
Additional EDA visual. VIF ranking from the Word draft. The strongest multicollinearity appears in density, pH, alcohol, and acidity-related features, which helps explain why coefficients in the GLM section require careful interpretation.

Feature scaling was applied only after the train-test split to prevent data leakage. That design choice matters because any transformation learned from the full dataset would leak test-set information into training and inflate the apparent quality of later model results. Keeping the scaling step downstream of the split preserves the independence of the held-out test set and makes the reported evaluation metrics more believable.

Boxplot Analysis of Key Chemical Features

The boxplot views from the Word draft reinforce two of the clearest EDA signals. Volatile acidity shows a visibly inverse relationship with wine quality: higher-quality wines cluster at lower volatile-acidity values, while lower-quality wines show both higher central values and wider spread. Alcohol shows the opposite pattern, rising steadily as wine quality increases and separating the better wines more clearly than most other single variables.

The boxplots also make the limits of univariate analysis visible. Even when the median trends are strong, there is still substantial overlap across adjacent quality groups, especially in the middle categories. That overlap helps explain why later models need to combine several features instead of relying on any one chemistry measurement alone.



Additional EDA visuals. The left panel shows the inverse relationship between volatile acidity and wine quality, while the right panel shows the positive relationship between alcohol and wine quality. Together they preserve the key boxplot evidence from the Word draft.

Statistical Analysis

This section presents the full statistical analysis of the wine quality dataset. Each subsection connects directly to the course module in which the concept was introduced and demonstrates how that concept applies to the wine quality prediction problem. All code was written in Python following the PEP 8 Style Guide (Van Rossum & Warsaw, 2001) using the pandas, numpy, scipy, matplotlib, and statsmodels libraries.

Descriptive Statistics

Module 1: Descriptive Statistics and Data Collection

As introduced in Module 1, descriptive statistics summarize the key characteristics of a dataset using measures of central tendency and dispersion. The mean represents the average value across all observations and the standard deviation measures how spread out values are from that mean. The five-number summary consisting of the minimum, first quartile, median, third quartile, and maximum gives a complete picture of each variable's distribution (Agresti & Kateri, 2022, p. 14).

The sample mean and standard deviation are calculated as follows:

$$\bar{x} = \frac{1}{n} \sum x_i \quad s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}}$$

Python Implementation

The following code computes descriptive statistics and generates the histogram visualization from the notebook:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

# Key variables for descriptive analysis
key_vars = ['alcohol', 'volatile_acidity', 'sulphates',
            'citric_acid', 'pH', 'quality']

# Five-number summary for all key variables
print(df[key_vars].describe().round(3))

# Mean comparison: Good quality vs Poor quality
good = df[df['quality_binary'] == 1]
poor = df[df['quality_binary'] == 0]

for var in key_vars[:-1]:
    print(f'{var}: Good_M={good[var].mean():.3f}, Poor_M={poor[var].mean():.3f}')

```

Table 2

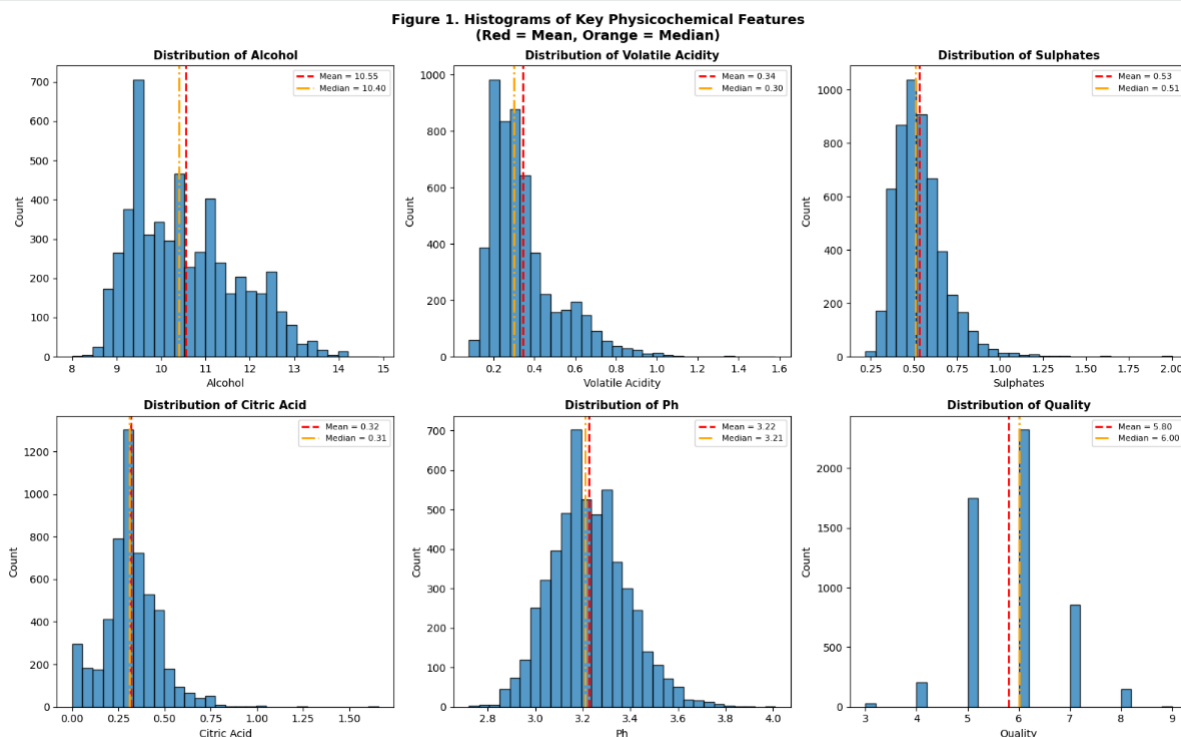
Descriptive Statistics for Key Physicochemical Variables by Quality Group

Variable	Mean (All)	SD (All)	Mean (Good)	Mean (Poor)
Alcohol (%)	10.549	1.186	10.929	9.913
Volatile Acidity (g/dm3)	0.344	0.168	0.309	0.403
Sulphates (g/dm3)	0.533	0.150	0.537	0.526
Citric Acid (g/dm3)	0.318	0.147	0.328	0.302
pH	3.225	0.160	3.229	3.217
Quality Score	5.796	0.880	6 or above	below 6

Note. Good quality is defined as a wine quality score of 6 or above. Poor quality is defined as a score below 6. Total n = 5,320 samples.

Figure 1

Histograms of Key Physicochemical Features with Mean and Median Lines.



Note. The red vertical line indicates the sample mean. Orange dashed line indicates the sample median. Right skewness is visible in volatile acidity and citric acid distributions.

The histograms in Figure 1 reveal that alcohol content and pH are approximately normally distributed, while volatile acidity and citric acid show right skewness, meaning most wines cluster at lower values with a long tail of outliers at the high end. When the mean exceeds the median, the distribution is right-skewed, which can be observed in the volatile acidity panel. The comparison in Table 2 shows that good quality wines have consistently higher alcohol content ($M = 10.929\%$, $SD = 1.192$) compared to poor quality wines ($M = 9.913\%$, $SD = 0.858$). Good wines also show lower volatile acidity at $M = 0.309$ g/dm³ versus $M = 0.403$ g/dm³ for poor wines. These differences provide the first statistical evidence that alcohol and volatile acidity are meaningful predictors of wine quality (Cortez et al., 2009, p. 550). Variables such as pH show much smaller group

differences, suggesting limited practical predictive value for quality classification.

Sampling Distribution and Central Limit Theorem

Module 3: Sampling Distributions and Simulations

The UCI Wine Quality Dataset represents a random sample from the population of all Vinho Verde wines produced in the region. As introduced in Module 3, a simple random sample is one in which every member of the population has an equal probability of selection, which minimizes sampling bias and allows generalization of findings to the broader wine population.

The Central Limit Theorem (CLT) states that regardless of the shape of the population distribution, the sampling distribution of the sample mean approaches a normal distribution as the sample size n increases (Agresti & Kateri, 2022, p. 87). Formally, if samples of size n are drawn from a population with mean μ and standard deviation σ :

$$\bar{x} \approx N\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$$

The term σ divided by the square root of n is the standard error (SE). It measures the precision of the sample mean as an estimator of the population mean. A smaller SE indicates a more precise estimate, achieved by increasing the sample size.

Python Implementation

```
import numpy as np
from scipy.stats import norm

# Population of alcohol content values
alcohol_pop = df['alcohol'].values
n_samples = 5000
sample_size = 50
```

```

# Draw 5000 random samples of size 50 and compute each mean
np.random.seed(42)
sample_means = [
    np.mean(np.random.choice(alcohol_pop, size=sample_size, replace=True))
    for _ in range(n_samples)
]

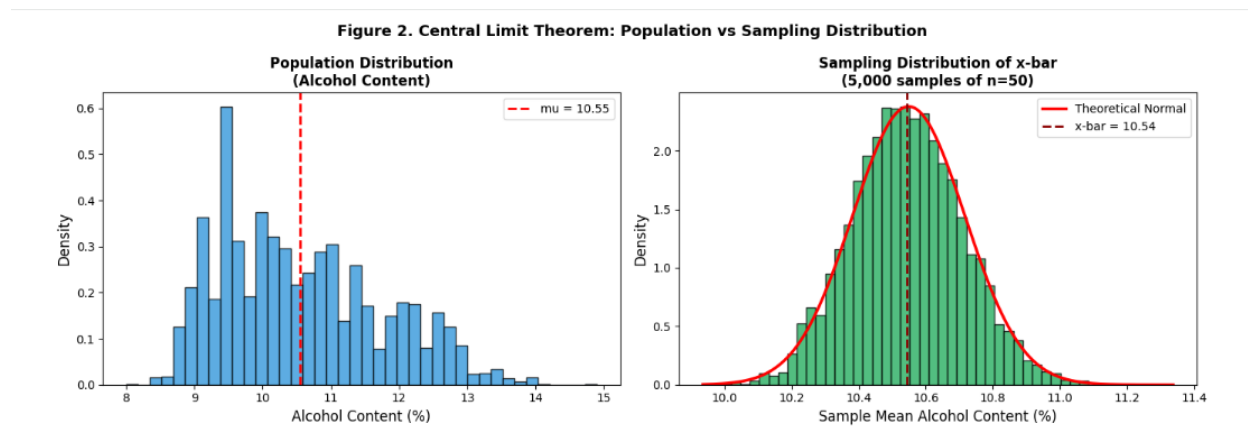
pop_mean = np.mean(alcohol_pop)
pop_std = np.std(alcohol_pop, ddof=1)
se = pop_std / np.sqrt(sample_size)

print(f'Population mean (mu) : {pop_mean:.4f}')
print(f'Standard Error (sigma/sqrt(n)) : {se:.4f}')
print(f'Mean of sampling distribution : {np.mean(sample_means):.4f}')
print(f'SD of sampling distribution : {np.std(sample_means):.4f}')

```

Figure 2

Central Limit Theorem: Population Distribution vs Sampling Distribution of the Mean.



Note. The left panel shows the slightly right-skewed population distribution of alcohol content. The right panel shows the approximately normal sampling distribution of the mean from 5,000 samples of $n = 50$, with the theoretical normal curve overlaid.

Figure 2 demonstrates the CLT empirically. Although alcohol content is slightly right-skewed in the population (left panel), the sampling distribution of the mean from 5,000 samples of size 50 is approximately normal (right panel), confirming the CLT (Agresti & Kateri, 2022, p. 89). The standard error of the mean is $SE = 0.168$, meaning that across repeated samples of size 50, the sample mean of alcohol content would fall within approximately 0.168 percent of the true population mean on average. This small standard error reflects the precision of the estimate given the large dataset size. This demonstration justifies the use of parametric statistical tests such as t-tests in subsequent sections, even when underlying variable distributions are not perfectly normal.

Confidence Intervals

Module 4: Inferential Statistics

A confidence interval provides a range of plausible values for a population parameter rather than a single point estimate. As taught in Module 4, the general formula for a confidence interval

around a sample mean is (Agresti & Kateri, 2022, p. 124):

$$CI = \bar{x} \pm z \times SE, \quad z \in \{1.00, 1.96, 2.576\}$$

The interval is interpreted as follows: there is 95% confidence that the true population mean lies within the computed interval. This does not mean there is a 95% probability the true mean is in any specific computed interval. The 95% refers to the long-run proportion of intervals that would contain the true mean across repeated sampling.

Python Implementation

```
def confidence_interval(data, confidence=0.95):
    """Calculate confidence interval for a sample mean.

    Args:
        data: Array of numerical values.
        confidence: Confidence level (0.68, 0.95, or 0.99).

    Returns:
        Tuple of (mean, lower bound, upper bound, standard error).
    """
    n = len(data)
    mean = np.mean(data)
    se = np.std(data, ddof=1) / np.sqrt(n)
    z_scores = {0.68: 1.000, 0.95: 1.960, 0.99: 2.576}
    z = z_scores[confidence]
    margin = z * se
    return mean, mean - margin, mean + margin, se
```

```
# Calculate 95% CIs for alcohol by quality group
for group_name, group_data in [('Good', good), ('Poor', poor)]:
    m, lo, hi, se = confidence_interval(group_data['alcohol'].values, 0.95)
    print(f'{group_name}: M={m:.3f}, 95% CI [{lo:.3f}, {hi:.3f}], SE={se:.4f}')
```

Table 3

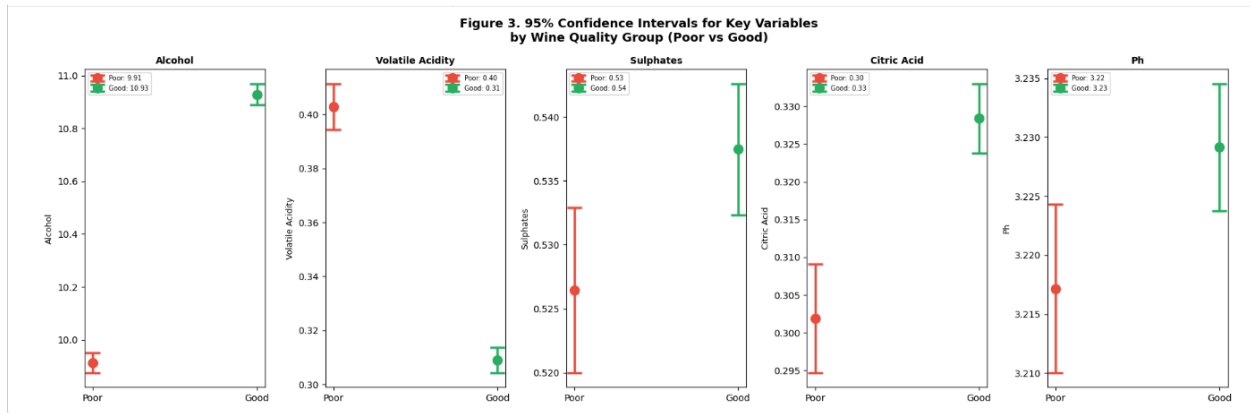
95% Confidence Intervals for Key Variables by Quality Group

Variable	Group	Mean	95% CI Lower	95% CI Upper	SE
Alcohol	Good	10.929	10.888	10.969	0.0206
Alcohol	Poor	9.913	9.875	9.951	0.0192
Volatile Acidity	Good	0.309	0.304	0.314	0.0024
Volatile Acidity	Poor	0.403	0.394	0.411	0.0043
Sulphates	Good	0.537	0.532	0.543	0.0026
Sulphates	Poor	0.526	0.520	0.533	0.0033

Note. Non-overlapping confidence intervals indicate statistically significant group differences.

Figure 3

95% Confidence Intervals for Key Variables by Wine Quality Group.



Note. Red markers indicate poor quality wines. Green markers indicate good quality wines.

Non-overlapping intervals for alcohol and volatile acidity confirm statistically significant group differences.

Figure 3 and Table 3 reveal that the 95% confidence intervals for alcohol content do not overlap between good and poor-quality wines: [10.888, 10.969] for good quality wines versus [9.875, 9.951] for poor quality wines. The gap between these intervals is approximately 0.93 percent alcohol, which is both statistically significant and practically meaningful. Similarly, volatile acidity and sulphates show separated intervals between groups, while pH remains much closer across classes and is therefore less useful in practical terms. The standard errors are very small for all variables because the sample size of 5,320 is large, meaning the estimates are highly precise. As taught in Module 4, a narrower confidence interval indicates a more precise estimate of the population's mean.

Hypothesis Testing and Confidence Intervals

This section builds directly on the earlier confidence-interval subsection by formally testing whether the observed differences between good and poor quality wines are statistically meaningful. Using a significance level of $\alpha = 0.05$, Welch two-sample t-tests, Pearson correlation tests, and a chi-square test of independence are applied to evaluate the main patterns in the dataset. Because the exploratory data analysis section already summarizes the distributional and correlation patterns, the role of this section is narrower: convert those patterns into formal inferential evidence.

To make the reasoning more explicit, each test can be interpreted as a comparison between observed structure in the sample and the random variation that would be expected under a null model. For the Welch two-sample test, the null and alternative hypotheses are

$$H_0 : \mu_{\text{Good}} = \mu_{\text{Poor}}, \quad H_A : \mu_{\text{Good}} \neq \mu_{\text{Poor}}.$$

The Welch test statistic standardizes the difference in sample means by the estimated sampling variability:

$$t = \frac{\bar{x}_{\text{Good}} - \bar{x}_{\text{Poor}}}{\sqrt{\frac{s_{\text{Good}}^2}{n_{\text{Good}}} + \frac{s_{\text{Poor}}^2}{n_{\text{Poor}}}}}.$$

Because the two groups do not need to have equal variances, the degrees of freedom are estimated with the Welch-Satterthwaite approximation,

$$v \approx \frac{\left(\frac{s_{\text{Good}}^2}{n_{\text{Good}}} + \frac{s_{\text{Poor}}^2}{n_{\text{Poor}}}\right)^2}{\frac{\left(\frac{s_{\text{Good}}^2}{n_{\text{Good}}}\right)^2}{n_{\text{Good}}-1} + \frac{\left(\frac{s_{\text{Poor}}^2}{n_{\text{Poor}}}\right)^2}{n_{\text{Poor}}-1}}.$$

This is helpful conceptually because a large absolute t value can happen only when the gap between the two sample means is large relative to the noise within the groups. The correlation and chi-square tests follow the same basic logic. For Pearson correlation, the null hypothesis is $H_0 : \rho = 0$, and the test statistic is

$$t = r\sqrt{\frac{n-2}{1-r^2}}.$$

For the contingency-table analysis, the expected count in each cell under independence is

$$E_{ij} = \frac{(\text{row total}_i)(\text{column total}_j)}{n},$$

and the chi-square test statistic is

$$\chi^2 = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}}.$$

Taken together, these formulas explain why the tiny p-values in this section are convincing. The observed wine-quality patterns are not only directionally different, but far enough from the null model that random sampling noise is not a believable explanation.

Independent Two-Sample t-Tests

The first set of tests compares the mean values of key physicochemical variables between wines classified as good quality and those classified as poor quality. Welch two-sample t-tests were used because they do not assume equal variances between groups. Across all five tested variables, the null hypothesis of equal means was rejected.

The clearest separation appears in alcohol and volatile acidity. Good wines had a mean alcohol content of 10.929 compared to 9.913 for poor wines, producing a very large test statistic ($t = 35.994$, $p = 3.29\text{e-}253$). Volatile acidity moved in the opposite direction, with good wines averaging 0.309 and poor wines averaging 0.403 ($t = -18.892$, $p = 1.35\text{e-}75$). Sulphates, citric acid, and pH were also statistically significant, although their effects were much smaller in practical terms. These results confirm that the mean differences observed earlier were not due to random sampling noise.

What makes the alcohol result especially strong is that the mean gap is large in both absolute and standardized terms. Even before moving to machine learning, the t-test already shows

that alcohol separates the two groups extremely well relative to the within-group spread. Volatile acidity shows the same kind of structure, but in the negative direction, meaning higher values are associated with lower quality outcomes.

Pearson Correlation Tests

The next step tested whether the linear relationships suggested by the exploratory analysis were statistically distinguishable from zero. Every feature examined produced a statistically significant correlation at $\alpha = 0.05$, but the strengths of those relationships differed meaningfully.

Alcohol showed the strongest positive relationship with quality ($r = 0.4694$, $p = 8.28\text{e-}290$), making it the clearest positive linear predictor in the dataset. Density showed the strongest negative overall relationship ($r = -0.3264$, $p = 2.56\text{e-}132$), while volatile acidity also had a substantial negative association ($r = -0.2652$, $p = 2.49\text{e-}86$). Chlorides and residual sugar were negatively related to quality as well, though more modestly. The main takeaway is that not all statistically significant variables are equally useful; alcohol, density, and volatile acidity carry much more practical signal than the weaker correlations.

The inferential importance of this subsection is that the directional relationships from the exploratory data analysis section are not just visible patterns in a chart. Given the sample size, the estimated correlations for alcohol, density, and volatile acidity are far too large relative to sampling error to dismiss as random noise. That result lets the later modeling sections treat those variables as substantively meaningful rather than merely suggestive.

Chi-Square Test of Independence

The final hypothesis test examines whether wine type and binary quality classification are independent. The chi-square test produced a statistic of 73.2011 with 1 degree of freedom and a p-value of $1.1709\text{e-}17$, so the null hypothesis of independence was rejected.

This means red and white wines do not share exactly the same good-versus-poor quality distribution in this sample. In the contingency table, white wines contributed a larger share of good

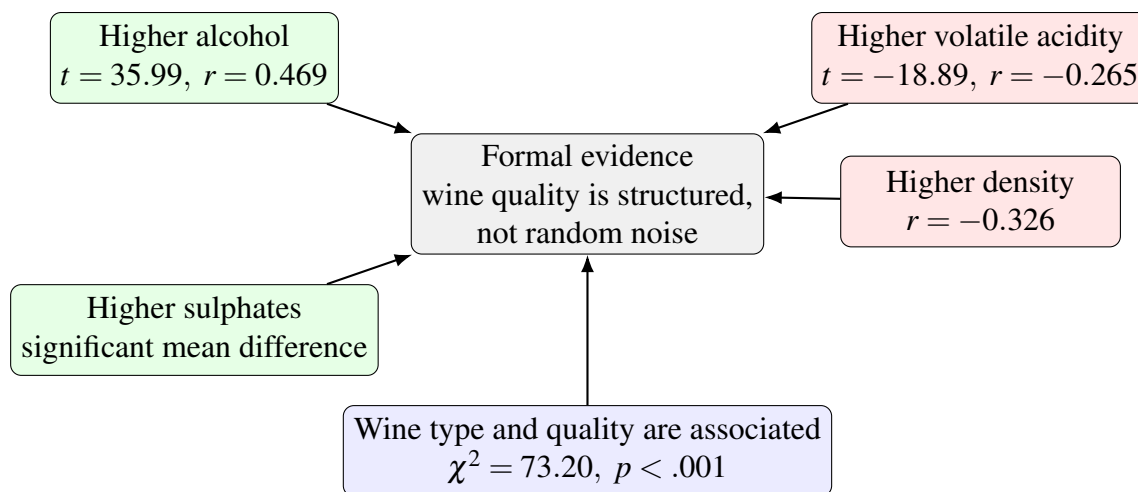
classifications than red wines. That result supports the decision to retain wine type as a predictor in the later classification models.

Mathematically, the interpretation is straightforward: if wine type and quality were independent, the observed counts would sit close to the expected counts computed from row and column totals alone. Instead, the deviations are much larger than chance would reasonably predict, so the model-building sections should preserve wine type as a meaningful input rather than treating it as irrelevant background information.

Section Summary

Taken together, the inferential results strongly support the same pattern seen in the descriptive statistics. Alcohol is the most important positive signal, while volatile acidity and density provide clear negative evidence about quality. These findings create the statistical foundation for the predictive modeling work that follows in the later modeling sections.

What makes this section persuasive is not any single p-value by itself, but the consistency of the evidence across multiple forms of inference. Mean comparisons, correlation tests, and contingency analysis all point toward the same basic story: the chemistry variables are not randomly related to quality. They contain real structure that later predictive models can exploit.

Figure 4*Inferential Evidence Map for the Hypothesis-Testing Section*

Note. This synthesis visual summarizes the formal inference results without repeating the exploratory figures. Left-side boxes indicate signals associated with better quality, right-side boxes indicate signals associated with lower quality, and the bottom box captures the chi-square association between wine type and quality classification.

The evidence map above makes the section more readable at a glance. Instead of scanning three separate test families independently, one can immediately see that alcohol and sulphates point toward higher quality, volatile acidity and density point away from it, and wine type still contributes meaningful categorical structure.

Bayesian Network*Module 5: Bayesian Networks*

Why a Bayesian Network

As introduced in Module 5, Bayesian Networks are directed acyclic graphs (DAGs) that encode probabilistic and causal relationships between variables through conditional independence assumptions. Unlike black-box machine learning models, Bayesian Networks allow the analyst to trace exactly why a prediction is made and to ask real-world probabilistic questions. The mathematical foundation is Bayes Theorem from Module 2 (Agresti & Kateri, 2022, p. 211):

$$P(\text{Quality} \mid \text{Features}) = \frac{P(\text{Features} \mid \text{Quality}) \times P(\text{Quality})}{P(\text{Features})}$$

The factorization property allows the joint distribution to be decomposed into a product of conditional distributions, reducing the number of parameters from exponential to manageable:

$$P(X_1, X_2, \dots, X_n) = \prod P(X_i \mid \text{Parents}(X_i))$$

This property makes Bayesian Networks particularly valuable in business settings where explainability is required. Production staff at a winery can understand and act on a probability output in a way they cannot interpret the internal weight matrices of a neural network.

Variable Discretization

Bayesian Networks require discrete variables. The continuous physicochemical features were binned into three categories of Low, Medium, and High. For alcohol content, Low is below 9.5%, Medium is 9.5% to 11.0%, and High is above 11.0%. Similar thresholds were applied to volatile acidity and sulphates. The binary quality target of Good or Poor serves as the network outcome node.

Python Implementation

```
# Discretize continuous features into Low, Medium, High categories
```

```

df_bn = df[['alcohol', 'volatile_acidity', 'sulphates', 'quality_binary']].copy()

df_bn['alcohol_bin'] = pd.cut(
    df_bn['alcohol'],
    bins=[0, 9.5, 11.0, 20],
    labels=['Low', 'Medium', 'High']
)

df_bn['vol_acid_bin'] = pd.cut(
    df_bn['volatile_acidity'],
    bins=[0, 0.35, 0.55, 2.0],
    labels=['Low', 'Medium', 'High']
)

df_bn['quality_label'] = df_bn['quality_binary'].map({0: 'Poor', 1: 'Good'})

# Bayes query function: P(Good Quality | evidence)
def bayes_query(data, conditions, target_col='quality_label', target_val='Good'):
    """Calculate P(target_val | conditions) using Bayes theorem.

    Args:
        data: Discretized DataFrame.
        conditions: Dict of column name to required value.
        target_col: Column to evaluate probability on.
        target_val: Target value to calculate probability for.

    Returns:
        Tuple of (probability float, sample count int).
    """
    mask = pd.Series([True] * len(data))

    for col, val in conditions.items():

```

```

        mask = mask & (data[col] == val)
    subset = data[mask]
    if len(subset) == 0:
        return 0.0, 0
    prob = (subset[target_col] == target_val).mean()
    return prob, len(subset)

# Compute prior and posterior probabilities
prior_good = (df_bn['quality_label'] == 'Good').mean()
print(f'Prior P(Good Quality): {prior_good:.3f}')

queries = [
    ({'alcohol_bin': 'High', 'vol_acid_bin': 'Low'}, 'HighAlc+LowVA'),
    ({'alcohol_bin': 'Low', 'vol_acid_bin': 'High'}, 'LowAlc+HighVA'),
]
for conditions, label in queries:
    prob, n = bayes_query(df_bn, conditions)
    print(f'{label}: P(Good) = {prob:.3f} [n={n}]')
```

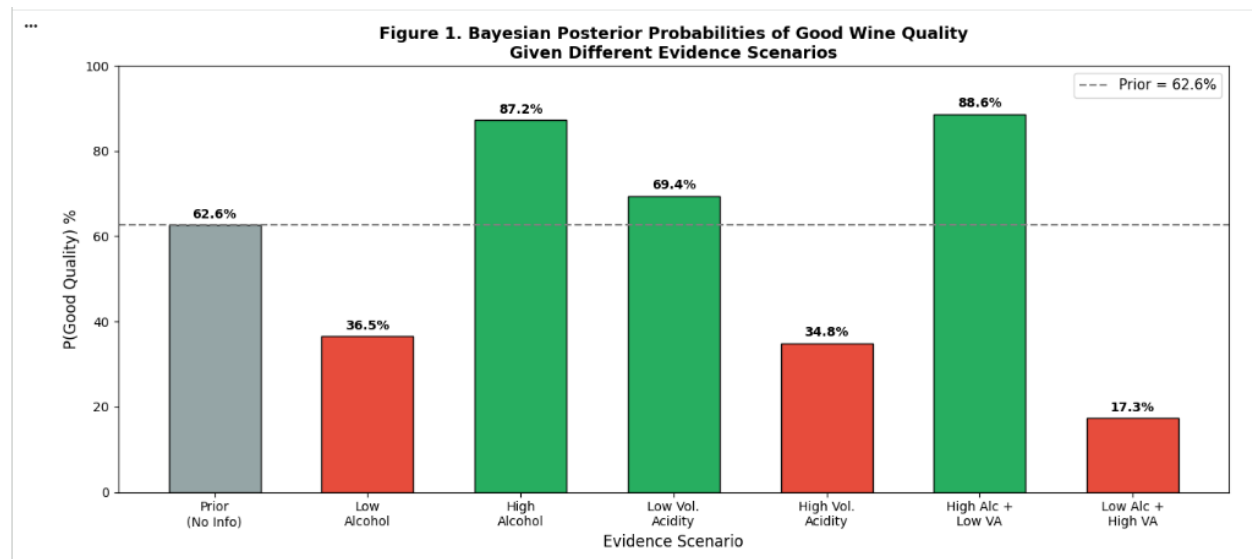
Posterior Probability Results

Starting from a prior probability of $P(\text{Good Quality}) = 0.626$, the Bayesian Network updates this estimate as chemical evidence is introduced. Table 4 presents the posterior probabilities for each evidence scenario.

Table 4*Bayesian Posterior Probabilities of Good Wine Quality Given Different Chemical Evidence**Scenarios*

Evidence Scenario	P(Good Quality)	Change from Prior
Prior (no information)	62.6%	Baseline
Low Alcohol only	36.5%	26.1 percentage points lower
High Alcohol only	87.2%	24.6 percentage points higher
Low Volatile Acidity only	69.4%	6.8 percentage points higher
High Volatile Acidity only	34.8%	27.8 percentage points lower
High Alcohol plus Low Volatile Acidity	88.6%	26.0 percentage points higher
Low Alcohol plus High Volatile Acidity	17.3%	45.3 percentage points lower

Note. Prior $P(\text{Good Quality}) = 0.626$ based on 5,320 cleaned samples.

Figure 4*Bayesian Posterior Probabilities of Good Wine Quality Given Different Evidence Scenarios.*

Note. Green bars indicate probabilities above the prior. Red bars indicate probabilities below the prior. The dashed horizontal line represents the prior probability of 62.6%.

Figure 4 and Table 4 reveal that the most powerful evidence scenario is the combination of high alcohol content and low volatile acidity, which raises the posterior probability of good quality classification to 88.6%. The least favorable scenario, low alcohol combined with high volatile acidity, drops it to 17.3%. This 71.3 percentage point range from worst to best case demonstrates that two inexpensive laboratory measurements can provide a highly informative early quality signal before any tasting panel evaluation is conducted.

From a business perspective this Bayesian framework functions like a real-time probability dashboard during fermentation monitoring. When a batch shows low alcohol developing alongside rising volatile acidity, production teams receive an early warning that the probability of good quality is dropping substantially below the 62.6% baseline. This allows corrective action such as adjusting fermentation temperature or yeast selection before the problem becomes irreversible.

Statistical Interpretation

The Bayesian approach connects directly to the course material from Module 2 (probability distributions) and Module 5 (Bayesian Networks). The prior probability of 62.6% reflects the marginal distribution of wine quality in the dataset. Each posterior query implements Bayes Theorem directly: the evidence (alcohol level, volatile acidity level) updates the posterior probability of good quality through the likelihood ratio $P(\text{Features} \mid \text{Quality})$ divided by $P(\text{Features})$. The factorization property of the network means that conditioning on multiple features simultaneously is computationally tractable even as the number of variables grows (Agresti & Kateri, 2022, p. 215).

Model Selection

This section compares predictive models using the 80/20 train-test split established in the notebook. The goal is not only to maximize predictive accuracy, but also to compare how classical linear models, Bayesian-style methods, and tree-based machine learning approaches perform on the same wine-quality problem. The emphasis here is selection rather than full diagnosis: top-line metrics

are used to choose a primary model and an interpretable companion model, while the detailed error analysis is deferred to the model-analysis section.

Before comparing models directly, it helps to state the evaluation criteria mathematically. For the regression models, the two main metrics are mean absolute error and the coefficient of determination:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

Lower MAE means the predicted wine scores are, on average, closer to the observed scores. Higher R^2 means the model explains more of the variation in quality. For classification, the most important metrics here are accuracy and F1:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad F_1 = \frac{2PR}{P + R},$$

where precision is $P = \frac{TP}{TP + FP}$ and recall is $R = \frac{TP}{TP + FN}$. Writing the metrics this way makes the tradeoff clearer: a model can achieve decent overall accuracy while still handling one class poorly, but a strong F1 score requires better balance between false positives and false negatives.

Why These Models Were Compared

These models were not chosen at random. The point of the comparison was to test several different ways of representing the same chemistry-to-quality relationship, beginning with the most interpretable linear models and then moving toward more flexible ensemble methods. Since this section had to answer both a predictive question and an explanation question, the model set was designed to vary along three dimensions at once: statistical assumptions, functional flexibility, and interpretability.

The first distinction is between regression and classification. In the regression setup, the model predicts the original numeric quality score. In the classification setup, the model predicts whether a wine is Good or Poor after binarization. This split is important because the two tasks

answer slightly different questions. Numeric prediction asks how close a model can get to the original score itself, while classification asks whether the model can support an operational yes-or-no quality screening decision.

Within each task, the models were chosen to form a ladder of complexity. The linear models ask whether the signal can be captured mostly through additive effects. The probabilistic baseline asks whether a very simple generative rule is already sufficient. The tree ensembles ask whether the data contain threshold behavior, interactions, or non-linear structure that simpler models cannot express cleanly. That ladder makes the final conclusion more convincing, because if several model families are tested and the same type of model repeatedly performs best, the selection is harder to dismiss as arbitrary.

Regression Models for Numeric Quality

For numeric quality prediction, Linear Regression was used as the baseline model because it is the clearest expression of the additive assumption. It writes the predicted score as

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip},$$

with the coefficients chosen to minimize the residual sum of squares,

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

This model was chosen not because linearity was expected to be fully correct, but because it provides the cleanest starting point for interpretation. If Linear Regression had performed nearly as well as the more complex models, then the chemistry-quality relationship could have been described mostly as a weighted additive rule. That would have made the later models less necessary.

Bayesian Ridge was chosen as the second regression baseline because the exploratory analysis already suggested multicollinearity among predictors such as alcohol, density, pH, and acidity-related variables. Bayesian Ridge keeps the same linear mean structure as ordinary regression, but

it places a prior distribution on the coefficients that shrinks unstable estimates toward zero. In practical terms, it penalizes coefficient magnitudes and reduces sensitivity to overlapping predictors. A convenient way to think about the fitted objective is

$$\min_{\beta} \left\{ \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

which is the same basic intuition as ridge regularization. It was included because if the main problem with Linear Regression was coefficient instability rather than linearity itself, then Bayesian shrinkage should have improved the results meaningfully.

Instead, Linear Regression and Bayesian Ridge landed in nearly the same range, each around $\text{MAE} = 0.548$ with R^2 near 0.325. That similarity is informative. It suggests that the central limitation is not just noisy coefficients, but the deeper fact that a single global linear equation does not capture all of the structure in the wine data.

Random Forest Regressor and Gradient Boosting Regressor were chosen to test that idea directly. Random Forest Regressor works by fitting many regression trees on bootstrap samples and then averaging their predictions:

$$\hat{y}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x),$$

where each $T_b(x)$ is a tree built on a slightly different resampled dataset and randomized feature subset. This model was chosen because it can learn threshold rules such as “quality rises once alcohol is above a certain level” or interaction rules such as “volatile acidity matters differently when alcohol is low versus high.” Averaging across many trees also makes it more stable than a single decision tree.

Gradient Boosting Regressor was chosen because it uses trees in a different way. Instead of averaging many independently grown trees, it builds an additive sequence of weak learners:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x),$$

where each new tree $h_m(x)$ is fitted to the residual structure left behind by the previous stage. This model was included because boosting is often strong on structured tabular data and sometimes outperforms bagging-style ensembles when small incremental corrections accumulate effectively.

The tree-based regressors performed better than the linear baselines. Random Forest improved the results to $\text{MAE} = 0.519$ and $R^2 = 0.392$, with Gradient Boosting close behind at $\text{MAE} = 0.520$ and $R^2 = 0.386$. Even though the numerical margin is not huge, the direction is consistent and meaningful: once the models are allowed to represent non-linear splits and interactions, performance improves. That makes the regression results important for this section because they already hint that wine quality is not governed by one smooth linear mechanism alone.

Classification Models for Good vs. Poor Quality

For binary classification, the test set contained 683 wines labeled Good and 381 labeled Poor, so the class balance was uneven but not extreme. Logistic Regression was selected as the main interpretable classification baseline because it models the probability of good quality directly through the logit link:

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}, \quad p_i = \frac{1}{1 + e^{-\eta_i}}.$$

This model was chosen because it connects naturally to the inferential parts of the paper. It transforms a linear predictor into a probability, produces coefficients with directional meaning, and makes it possible to explain how a feature such as alcohol or volatile acidity moves the odds of a wine being classified as Good. Its main strength is interpretability. Its main weakness is that it still imposes one global linear decision boundary in predictor space.

Gaussian Naive Bayes was chosen as a contrasting probabilistic baseline. Instead of fitting one linear boundary directly, it uses Bayes' rule and predicts the class with the largest posterior score

$$\hat{y} = \arg \max_c P(Y = c) \prod_{j=1}^p f_j(x_j | Y = c),$$

where the Gaussian version assumes each class-conditional feature density f_j is normal. This model was included because it is simple, fast, and conceptually different from Logistic Regression. It answers the useful question, “How well can a very lightweight probabilistic classifier do if each feature contributes mostly on its own?” In this dataset, that assumption is too strong. The EDA and VIF analysis already showed that alcohol, density, sulfur dioxide, and acidity-related variables are correlated, so Naive Bayes was expected to act more like a benchmark than a likely winner.

The ensemble classifiers were chosen to test whether non-linear boundaries and interactions would improve the binary decision problem in the same way they improved regression. Random Forest Classifier works by aggregating votes across many trees,

$$\hat{y}_{RF}(x) = \text{majority vote}\{T_1(x), \dots, T_B(x)\},$$

which makes it well suited for interaction-heavy problems where one chemistry variable changes meaning depending on the level of another. It was chosen because the earlier sections repeatedly suggested threshold behavior: for example, alcohol is helpful, but likely not in exactly the same way at every level of volatile acidity or density.

Gradient Boosting Classifier was chosen because it represents a second, distinct ensemble logic. Instead of stabilizing predictions by averaging many trees, it builds a stagewise additive decision rule in which each new tree focuses on the observations that earlier trees handled poorly. In classification terms, that means the model keeps refining the boundary around difficult cases, especially borderline wines whose chemistry does not fit a simple pattern. It was included because boosting is often very competitive on tabular classification tasks and provides a useful comparison against Random Forest’s bagging-based approach.

The classification results fit the same pattern seen on the regression side. Logistic Regres-

sion remained a strong interpretable baseline with Accuracy = 0.755 and weighted F1 = 0.751, which means a linear log-odds rule already captures a substantial share of the signal. Gaussian Naive Bayes dropped to Accuracy = 0.680 and weighted F1 = 0.677, which is consistent with the failure of the independence assumption. The ensemble methods formed the top tier again, with Random Forest posting the best overall classification performance and Gradient Boosting finishing a very close second. What matters most is not only that Random Forest ranked first, but that the entire ranking supports the same structural conclusion: the wine-quality classification problem contains real non-linear interactions that simpler parametric models only approximate.

Model Selection Decision

Based on the top-line results across both tasks, Random Forest is the strongest overall predictive model in this project. It ranked first for regression and first for classification, which makes it the most defensible choice as the primary prediction model. In practical terms, Random Forest is the model most appropriate when the question is, “Given this chemistry profile, what should be predicted?” It gives the best overall balance of flexibility, robustness, and empirical performance.

The reason this choice is stronger than a simple scoreboard argument is that the ranking also matches the structure of the data. Earlier sections showed skewness, outliers, correlation among predictors, and repeated signs of non-linear behavior. A model that partitions the predictor space recursively and averages over many trees is naturally better suited to that environment than one model-wide linear equation or one independence-based probability factorization. In other words, Random Forest was not chosen only because it scored highest; it was chosen because the way it learns is consistent with what the earlier statistical sections already implied about the data.

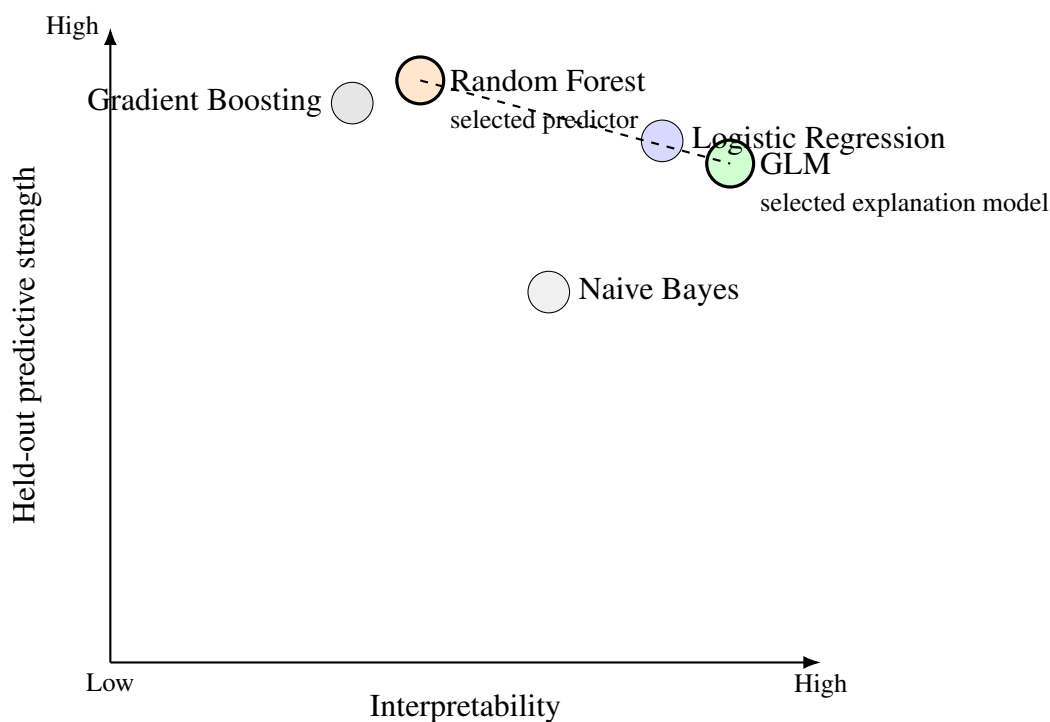
At the same time, Logistic Regression and the GLM remain essential because prediction and explanation are not identical goals. Random Forest is better at absorbing complex interactions, but Logistic Regression is easier to interpret because its coefficients show how the predictors move the odds of good quality in a direction that a reader can explain. That makes the GLM framework valuable whenever the question becomes, “Why did the model lean toward good quality?” rather

than just, “What class should be predicted?”

The final decision is therefore a two-model conclusion rather than a one-model compromise. Random Forest was chosen over Gradient Boosting because it finished slightly ahead while staying strong in both tasks. It was chosen over Linear Regression, Bayesian Ridge, Logistic Regression, and Naive Bayes because those simpler models each answer an important benchmark question, but they leave meaningful predictive performance on the table. The overall recommendation is to use Random Forest as the main prediction engine and to use the GLM interpretation when communicating the statistical meaning of the result. That pairing makes the model-selection section stronger because it shows both how the models work and why the final choice follows logically from the behavior of the data.

Figure 5

Performance-Interpretability Tradeoff Behind the Final Model-Selection Decision



Note. Horizontal position reflects qualitative interpretability and vertical position reflects held-out predictive performance summarized from the classification and regression results. The dashed connection highlights the final pairing used in this analysis: Random Forest for prediction and the GLM for explanation.

The tradeoff map above shows why a single-model answer would be less convincing than a paired-model strategy. Random Forest sits highest on performance, while the GLM sits furthest to the right on interpretability, so keeping both models in the final story is more defensible than forcing one method to satisfy both goals equally well.

Generalized Linear Model (GLM)

Module 6: Generalized Linear Models

Why a GLM Instead of OLS

As explained in Module 6, Ordinary Least Squares (OLS) regression assumes a continuous, normally distributed response variable. The target variable in this study is binary (Good = 1, Poor = 0). Applying OLS to a binary outcome produces predicted values outside the [0, 1] range, which cannot be meaningfully interpreted as probabilities. The appropriate solution is a Generalized Linear Model (GLM) with a binomial family and logit link function, commonly referred to as logistic regression (Agresti & Kateri, 2022, pp. 299-302).

A GLM has three components: (a) the random component specifying the probability distribution of the response variable (binomial for binary outcomes), (b) the systematic component which is a linear predictor combining the independent variables, and (c) the link function connecting the expected value of the response to the linear predictor. For logistic regression the logit link function is:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Solving this equation for p gives the sigmoid function, which maps any real-valued linear combination to a probability between 0 and 1. Parameters are estimated through Maximum Likelihood Estimation (MLE) rather than least squares, finding the parameter values that maximize the probability of observing the actual binary outcomes in the training data. The odds ratio $\exp(B)$ for each predictor quantifies how a one-unit increase in that variable changes the odds of good quality classification, holding all other variables constant.

Model Fitting and Implementation

The final notebook fits the GLM with the statsmodels library using the prepared feature matrix and then evaluates the resulting probabilities on the held-out test subset defined earlier in the workflow. Nine physicochemical features were used as predictors. The following code mirrors the final notebook implementation:

```
import statsmodels.api as sm
import numpy as np
from sklearn.metrics import accuracy_score, roc_auc_score, precision_score
from sklearn.metrics import recall_score, f1_score

# Define features for the GLM
glm_features = [
    'alcohol', 'volatile_acidity', 'sulphates', 'citric_acid',
    'pH', 'chlorides', 'density', 'residual_sugar', 'wine_type_enc'
]

# Fit GLM using the prepared full feature matrix
X_glm = sm.add_constant(df[glm_features])
y_glm = df['quality_binary']

glm_model = sm.GLM(y_glm, X_glm, family=sm.families.Binomial())
glm_results = glm_model.fit()
print(glm_results.summary())

# Evaluate on the held-out test subset
X_test_for_glm = df.loc[X_test.index, glm_features]
X_test_glm = sm.add_constant(X_test_for_glm)
glm_probs = glm_results.predict(X_test_glm)
```

```

y_test_binary = df.loc[X_test.index, 'quality_binary']
glm_preds = (glm_probs >= 0.5).astype(int)

print(f'Accuracy_{_}:_{accuracy_score(y_test_binary, _glm_preds):.4f}'))
print(f'AUC-ROC_{_}:_{roc_auc_score(y_test_binary, _glm_probs):.4f}'))
print(f'Precision_{_}:_{precision_score(y_test_binary, _glm_preds):.4f}'))
print(f'Recall_{_}:_{recall_score(y_test_binary, _glm_preds):.4f}'))
print(f'F1_{_}Score_{_}:_{f1_score(y_test_binary, _glm_preds):.4f}'))

```

Coefficient Table and Odds Ratios

Most of the predictors were statistically significant in the fitted GLM, though not all of them remained significant after controlling for the others. Table 6 presents the coefficient estimates and their corresponding odds ratios.

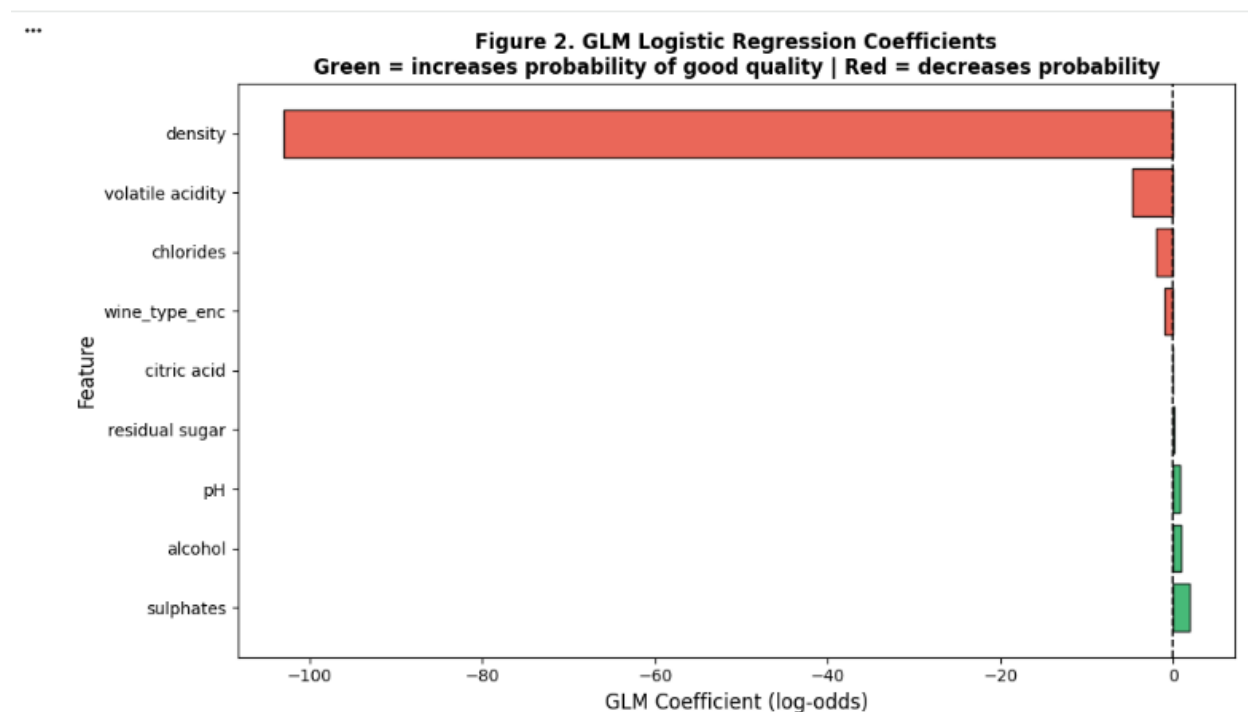
Table 6*GLM Logistic Regression Coefficient Table with Odds Ratios*

Variable	Coefficient	Odds Ratio	Direction and Interpretation
Alcohol	+0.8676	2.3812	Each 1 % increase more than doubles the odds of good quality
Volatile Acidity	-4.7804	0.0084	Each 1 g/dm ³ increase reduces odds to less than 1% of previous value
Sulphates	+1.9004	6.6883	Each 1 g/dm ³ increase raises odds by more than 6 times
Citric Acid	+0.0047	1.0047	Minimal effect — not statistically significant ($p = 0.987$)
pH	+0.8318	2.2975	Higher acidity increases quality probability
Residual Sugar	+0.0908	1.0950	Small positive effect
Chlorides	-1.9388	0.1439	Negative estimate, but not statistically significant at $p < .05$
Density	-103.037	less than 0.001	Strong negative effect on quality
Wine Type (Red = 0)	-0.9760	0.3768	Red wine has lower baseline probability

Note. Alcohol, volatile acidity, sulphates, pH, residual sugar, density, and wine type are statistically significant in the notebook output. Citric acid ($p = .9865$) and chlorides ($p = .0748$) are not. Odds Ratio = $\exp(\text{Coefficient})$. Coefficients represent log-odds.

Figure 5

GLM Logistic Regression Coefficients Sorted by Effect Size.



Note. Green bars indicate variables that increase the log-odds of good quality classification. Red bars indicate variables that decrease the log-odds.

Figure 5 and Table 6 reveal that sulphates has the highest positive odds ratio ($OR = 6.688$), meaning each one unit increase multiplies the odds of good quality classification by more than six times. Alcohol ($OR = 2.381$) confirms the pattern identified in the Bayesian Network analysis: higher alcohol content strongly increases the probability of good quality. Volatile acidity has the most dramatic negative effect ($OR = 0.0084$), meaning a one-unit increase reduces the odds of good quality to less than 1% of their previous value. This reflects the well-known association between high acetic acid levels and poor wine quality, where acetic acid is responsible for the vinegar smell that tasters penalize heavily (Cortez et al., 2009, p. 551).

The density coefficient is extremely large and negative, which warrants careful interpretation. Density is highly correlated with alcohol content and residual sugar (multicollinearity). In the

presence of these correlated predictors, density absorbs a disproportionately large coefficient. This is a known limitation of logistic regression in the presence of multicollinearity and is one reason why tree-based models that handle feature interactions implicitly may be preferred for prediction accuracy, even if they sacrifice interpretability.

Model Performance on Test Set

The fitted GLM was evaluated on the held-out test set (20% of data, approximately 1,064 samples) not used during model training. Evaluating unseen data is essential for assessing generalization performance. Table 7 presents all five evaluation metrics.

Table 7

GLM Performance Metrics on Held-Out Test Set

Metric	Value	Interpretation
Accuracy	0.7556	75.6% of wines correctly classified as Good or Poor
AUC-ROC	0.8250	82.5% better than random guessing at any threshold
Precision	0.7909	79.1% of wines predicted as Good are actually Good
Recall	0.8419	84.2% of all actual Good wines are correctly identified
F1 Score	0.8156	Harmonic mean of precision and recall

Note. All metrics reported on the held-out test set not used during model training ($n = 1,064$ samples). AUC-ROC = Area Under the Receiver Operating Characteristic curve.

The AUC-ROC of 0.8250 indicates strong discriminative ability. The model can distinguish between good and poor quality wines with 82.5% reliability across all possible classification thresholds. The recall of 0.8419 means the model correctly identifies 84.2% of all actual good quality wines. For a winery, high recall is particularly important because a false negative (incorrectly flagging a good quality wine as poor) results in lost revenue when that wine is underpriced

or discarded. The precision of 0.7909 indicates that about 79.1% of wines predicted as good are actually good, meaning the false positive rate is manageable for production-scale decisions.

Comparison with Classification Models

Comparing the GLM (AUC = 0.8250, Accuracy = 0.7556) against the tree-based models from the model-selection section shows that Random Forest achieved the highest classification accuracy (0.7829) and Gradient Boosting the second highest (0.7801). The GLM performs competitively while offering a major advantage that tree models cannot match: interpretability. Each coefficient in the GLM has a direct business interpretation through its odds ratio, allowing winery management to understand exactly which chemical properties drive the quality prediction and by how much. This interpretability is a critical requirement in regulated industries where model decisions must be explainable to non-technical stakeholders and auditors (Agresti & Kateri, 2022, p. 303).

Model Analysis

This section interprets the fitted-model results after the model-selection step described earlier. Rather than re-arguing which model ranked first, the goal here is to examine how the models behaved on the held-out test set, what kinds of mistakes they made, and what those patterns imply for practical wine-quality prediction.

Evaluation Framework

The project evaluated the wine-quality task in two parallel ways. In the regression formulation, the models predicted the original numeric quality score. In the classification formulation, wines with quality scores of 6 or higher were labeled Good and wines below 6 were labeled Poor. All models were evaluated on an 80/20 train-test split. Classification performance was summarized using accuracy, weighted F1, and AUC-ROC, while regression performance was summarized using mean absolute error (MAE) and the coefficient of determination (R^2).

Table 8*Classification Performance Summary from the Model Analysis*

Model	Accuracy	Weighted F1	AUC-ROC
Logistic Regression	0.755	0.751	0.825
Gaussian Naive Bayes	0.680	0.677	N/A
Random Forest Classifier	0.783	0.780	0.850
Gradient Boosting Classifier	0.780	0.778	N/A

Note. Values are aligned to the final notebook's saved model-comparison outputs.

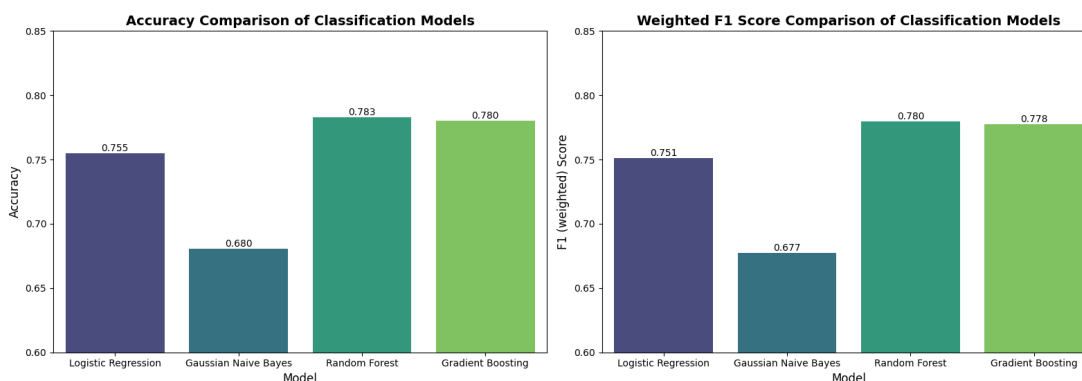
Table 9*Regression Performance Summary from the Model Analysis*

Model	MAE	R^2
Linear Regression	0.548	0.325
Bayesian Ridge	0.548	0.324
Random Forest Regressor	0.519	0.392
Gradient Boosting Regressor	0.520	0.386

Note. Values are aligned to the final notebook's saved model-comparison outputs. Lower MAE and higher R^2 indicate better regression performance.

These summary tables reinforce the conclusion from the model-selection section that ensemble tree methods outperform the linear and probabilistic baselines. What this section adds is a closer interpretation of why that ranking holds up once the actual error patterns are inspected.

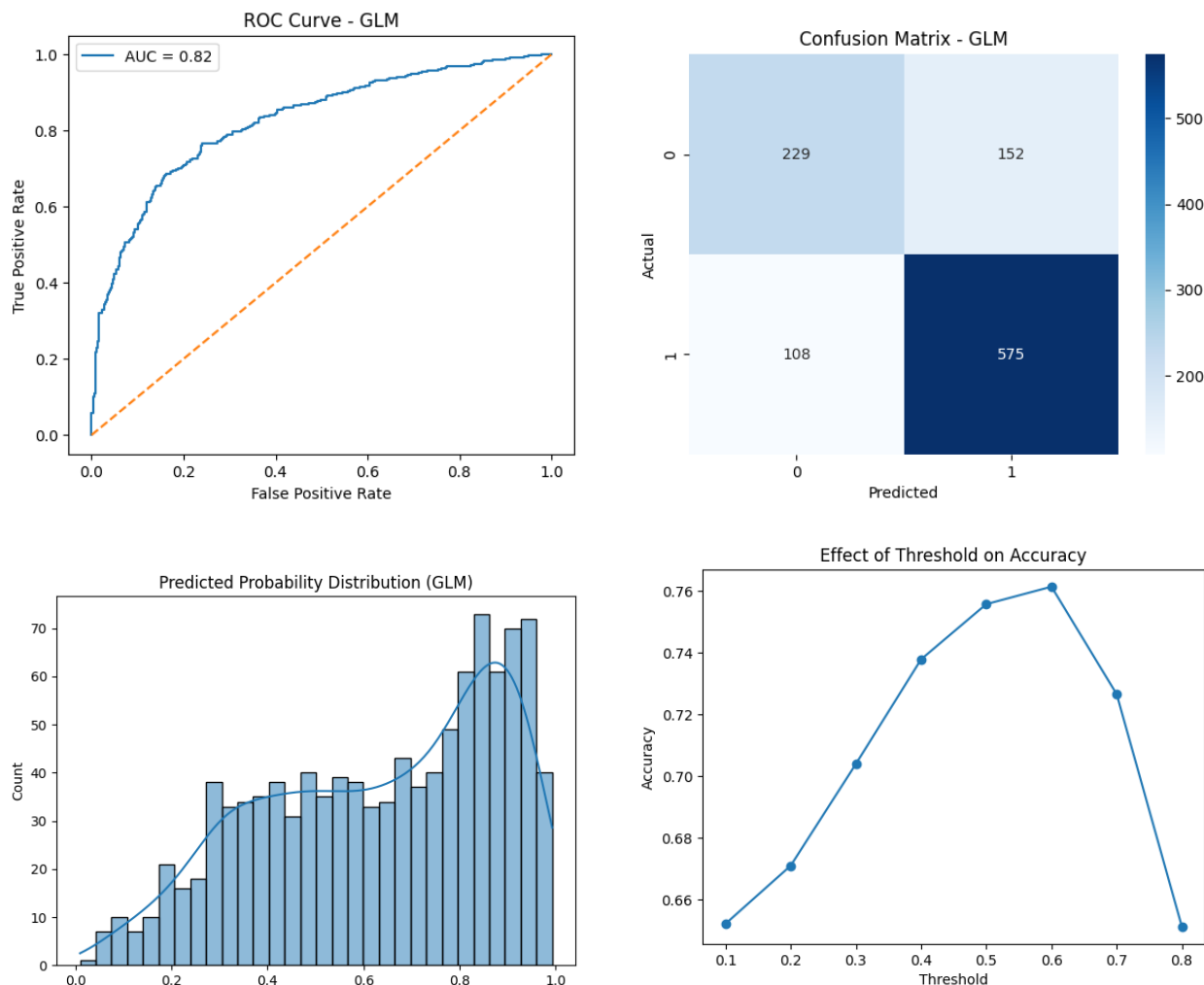
Figure 7. Classification Model Performance Comparison



Additional model-analysis visual. This side-by-side comparison from the Word draft preserves the classification performance chart for the LaTeX version. It makes the same ranking as Table 8 visible at a glance: Random Forest first, Gradient Boosting second, Logistic Regression third, and Gaussian Naive Bayes last.

Classification Diagnostics

Logistic Regression provided a strong interpretable baseline, reaching accuracy of 0.755, weighted F1 of 0.751, and AUC-ROC of approximately 0.825. Roopleen's confusion-matrix review reported 575 true positives, 229 true negatives, 152 false positives, and 108 false negatives. That mix shows a model that already separates the two quality classes reasonably well, but still makes enough mistakes near the decision boundary to leave room for improvement. The threshold-sensitivity analysis suggested that the GLM performed best across roughly the 0.5 to 0.6 range, with a peak near 0.6. At the same time, multicollinearity among predictors such as density, pH, alcohol, and fixed acidity likely spread influence across correlated variables and reduced interpretive clarity.



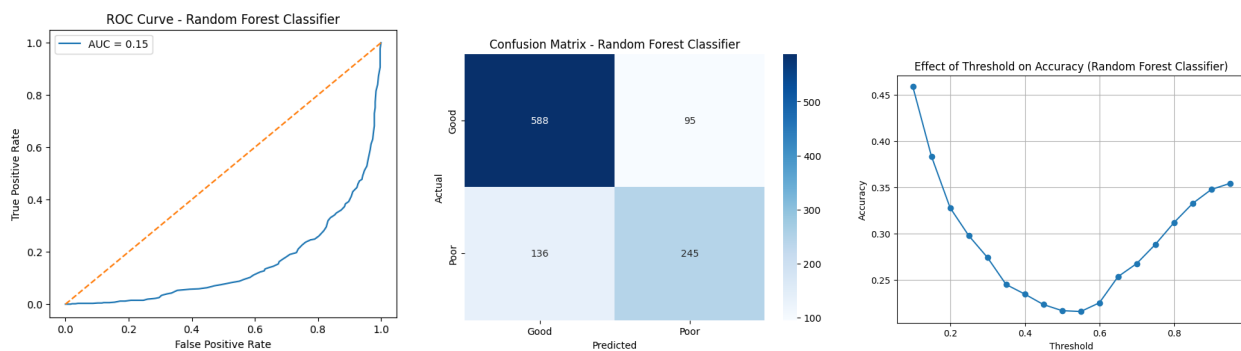
Additional GLM diagnostics from the Word draft. These four views preserve the ROC curve, confusion matrix, predicted-probability histogram, and threshold-sensitivity plot for the GLM. Together they show a model that is meaningfully better than random, reasonably balanced across the two classes, and most stable around the midrange classification thresholds.

The ROC curve confirms that the GLM separates the two classes well, while the confusion matrix shows a fairly balanced tradeoff between false positives and false negatives. The predicted-probability distribution also helps interpret how the model behaves: many wines receive high good-quality probabilities, but there is still a broad middle band where chemistry profiles overlap and classification is less certain. The threshold plot in turn shows why a cutoff near 0.5 to 0.6 is

operationally sensible for this model.

Gaussian Naive Bayes performed worst among the classifiers, with accuracy of 0.680 and weighted F1 of 0.677. This weaker performance is consistent with the exploratory evidence that several physicochemical variables are correlated. Because Naive Bayes assumes conditional independence among predictors, it is a poor fit for a dataset where sulfur dioxide measures, alcohol, density, and acidity-related features interact substantially.

Random Forest delivered the strongest classification performance, with accuracy of 0.7829, precision of 0.8122, recall of 0.8609, F1 of 0.8358, and AUC-ROC of 0.8496. Roopleen's confusion matrix for the Random Forest reported 588 true positives, 245 true negatives, 136 false positives, and 95 false negatives, which is an improvement over the GLM across both classes and especially valuable in reducing missed good-quality wines. The threshold-sensitivity analysis was also more stable than the GLM, with strong performance sustained across approximately the 0.5 to 0.65 range. That wider stability band matters in practice because it means the classifier is less fragile when the business decision threshold changes.



Additional Random Forest diagnostics from the Word draft. The ROC curve, confusion matrix, and threshold plot show why Random Forest became the preferred classifier. Compared with the GLM, it maintains a stronger separation curve, fewer classification errors overall, and a wider band of usable threshold choices.

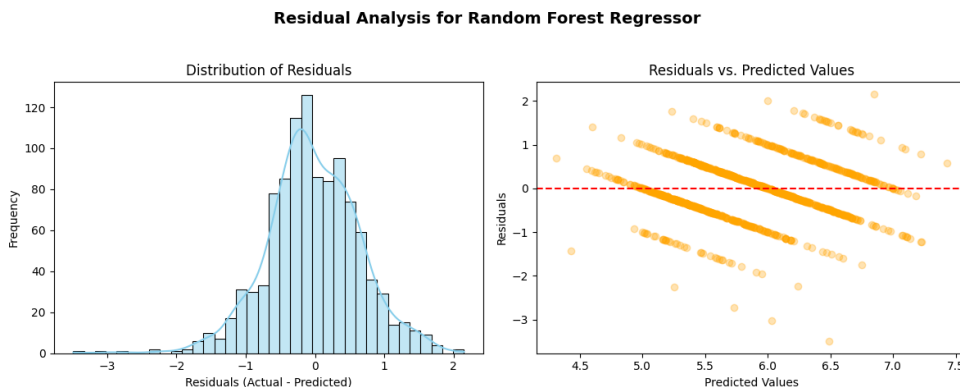
Roopleen's feature-importance discussion further clarified why Random Forest works so

well. Alcohol emerged as the most important predictor, followed by volatile acidity and sulphates, with density also contributing meaningfully. Residual sugar, pH, and chlorides were less influential in the final ensemble. This ordering is consistent with the earlier exploratory and inferential findings, which strengthens the overall coherence of the paper.

Regression Diagnostics

The regression results tell a similar story. Linear Regression and Bayesian Ridge formed useful baselines, but each explained only about one-third of the variation in wine quality. Random Forest Regressor achieved the best numeric performance with $MAE = 0.519$ and $R^2 = 0.392$, narrowly outperforming Gradient Boosting Regressor. Even though the regression gains are modest in absolute size, they are consistent, which suggests that the chemistry-quality relationship includes non-linear structure that simple additive models do not fully capture.

Roopleen's residual analysis of the Random Forest Regressor adds an important diagnostic layer. The residual histogram was centered close to zero, indicating that the model was not systematically overpredicting or underpredicting wine quality. The residual-versus-predicted scatterplot showed no strong funnel shape or obvious curvature, which supports the idea that variance remained relatively stable across the prediction range. A few larger residuals still appeared for atypical wines, but the overall pattern was sufficiently well behaved to support the use of Random Forest as the most reliable regression model in the project.



Additional regression diagnostic from the Word draft. The residual histogram is centered near zero and the residual-versus-predicted scatterplot shows no strong structure, which supports the conclusion that the Random Forest regressor captures the main non-linear signal without introducing obvious systematic bias.

Cross-Model Interpretation

Taken together, the model-analysis results show that Random Forest consistently outperforms the linear and probabilistic alternatives because it can adapt to non-linear interactions among acidity, density, sulphates, sulfur dioxide, and alcohol. The GLM remains valuable as an explanation model because its coefficients can still be interpreted directly, but its performance is constrained by the same multicollinearity and linearity limits that appeared earlier in the exploratory analysis. Gaussian Naive Bayes, by contrast, underperforms largely because its independence assumptions are not compatible with the observed chemistry relationships. The broad conclusion of this section is therefore not just that Random Forest scores highest, but that the structure of the data itself favors flexible ensemble methods over simpler parametric rules.

Conclusion and Recommendations

Summary of Findings

This project asked whether physicochemical measurements can be used to predict wine quality, and the overall answer is yes. Across descriptive statistics, hypothesis testing, Bayesian reasoning, and predictive modeling, the same variables repeatedly emerged as the clearest signals: alcohol is positively associated with quality, while volatile acidity is negatively associated with it. The cleaned dataset contained 5,320 wines, of which 62.6% were classified as good quality, providing a large enough sample to support both inference and model evaluation.

The inferential results showed that good and poor wines differ significantly across the key chemical variables, especially alcohol and volatile acidity. The Bayesian Network reached a similar practical conclusion: wines with high alcohol and low volatile acidity had an 88.6% posterior probability of being good quality, while wines with low alcohol and high volatile acidity dropped to 17.3%. The model-selection results then showed that Random Forest was the strongest predictive model for both regression and classification.

The GLM remained valuable because it provided interpretable coefficients and odds ratios while still performing well on held-out data. In the final notebook output, it achieved Accuracy = 0.7556, AUC-ROC = 0.8250, Precision = 0.7909, Recall = 0.8419, and F1 = 0.8156. That makes it a strong explanation model even though Random Forest remains the best pure predictor.

What makes the overall argument stronger is the convergence of the methods rather than the dominance of any one model. Descriptive statistics identified the basic direction of the signal, formal inference confirmed that the differences were not random, the Bayesian Network translated those patterns into probabilities, and the machine learning models turned them into high-performing predictions. That step-by-step agreement is what makes the final conclusion feel dependable rather than overfitted to a single technique.

Quantitative Decision Rule

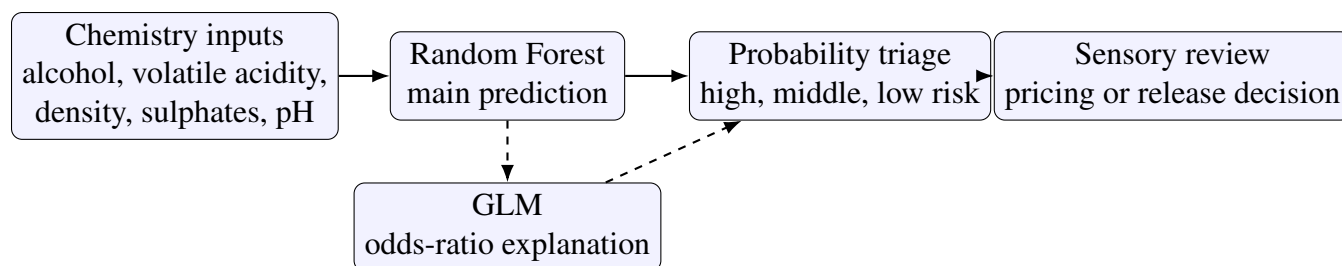
One way to convert the project into an operational rule is to treat the final model output as an estimated probability

$$\hat{p} = P(Y = 1 \mid \mathbf{x}),$$

where $Y = 1$ means Good Quality and $\mathbf{x}$ represents the physicochemical measurements for a wine. A simple three-band triage policy can then be written as

$$\hat{p} \geq \tau_H \Rightarrow \text{priority candidate}, \quad \tau_L < \hat{p} < \tau_H \Rightarrow \text{manual review}, \quad \hat{p} \leq \tau_L \Rightarrow \text{risk flag}.$$

This framing is useful because it turns the statistical findings into an actual decision system instead of leaving them as isolated test results. The thresholds τ_L and τ_H can be set by the winery based on how costly false positives and false negatives are. If the business wants to avoid missing potentially strong wines, it can choose a lower high-quality threshold. If it wants to avoid overrating weaker wines, it can raise that threshold and send more borderline cases to human review.

Figure 6*Operational workflow*

Note. Random Forest acts as the primary predictor, while the GLM acts as an explanation layer that helps justify why a batch received a high- or low-quality probability. Borderline cases can be routed to human tasters instead of forcing a fully automated yes-or-no decision.

Figure 6 shows why the conclusion does not have to be “machine learning replaces tasting.” A better interpretation is that chemistry-based prediction can narrow the decision space, surface the highest-value candidates, and flag risky batches earlier. That is a more realistic and defensible use of the models in a production setting.

Recommendations

1. Use alcohol and volatile acidity as primary screening variables. These two measurements were the most consistent predictors across the inferential tests, Bayesian analysis, and machine learning models.

2. Deploy Random Forest as the main operational model. It achieved the strongest overall performance in both numeric scoring and binary classification, making it the best single framework for prediction.

3. Keep the GLM as an interpretation and audit model. Even when it is slightly less accurate than Random Forest, it is much easier to explain to a non-technical audience and can show how each variable shifts the odds of good quality.

4. Use probability-based triage before sensory review. Wines predicted as strong good-quality candidates can be prioritized for premium evaluation, while borderline cases can be routed for additional tasting or process review.

5. Improve future models with richer data. Chemistry alone explains a meaningful share of quality, but vineyard conditions, fermentation process variables, and sensory context would likely improve performance further.

Final Conclusion

Overall, the project supports the feasibility of an evidence-based wine quality screening system. Physicochemical measurements do not fully replace human tasting, but they can substantially improve consistency, speed, and early-stage decision making. The most defensible practical strategy is to combine Random Forest for prediction, the GLM for interpretability, and alcohol-plus-volatile-acidity monitoring as the clearest laboratory-based indicators of quality.

From a mathematical standpoint, the strongest conclusion is that several independent tools all pointed in the same direction. When descriptive summaries, test statistics, posterior probabilities, and predictive models converge, confidence in the result increases. That convergence is what makes the final recommendation feel justified rather than simply convenient.

References

- Agresti, A., & Kateri, M. (2022). *Foundations of statistics for data scientists: With R and Python*. CRC Press.
- Ankan, A., & Panda, A. (2015). pgmpy: Probabilistic graphical models using Python. *Proceedings of the 14th Python in Science Conference*, 6–11. <https://pgmpy.org>
- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547–553. <https://doi.org/10.1016/j.dss.2009.05.016>

Module 1 lecture and lab: Descriptive statistics and data collection. (2025). *AAI-500 course materials*. University of San Diego.

Module 2 lecture and lab: Probability distributions. (2025). *AAI-500 course materials*. University of San Diego.

Module 3 lecture and lab: Sampling distributions and simulations. (2025). *AAI-500 course materials*. University of San Diego.

Module 4 lecture and lab: Inferential statistics. (2025). *AAI-500 course materials*. University of San Diego.

Module 5 lecture and lab: Bayesian networks. (2025). *AAI-500 course materials*. University of San Diego.

Module 6 lecture and lab: Generalized linear models. (2025). *AAI-500 course materials*. University of San Diego.

Module 7 lecture and lab: Classification, clustering, and linear algebra. (2025). *AAI-500 course materials*. University of San Diego.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*, 57–61. <https://doi.org/10.25080/Majora-92bf1922-011>

UCI Machine Learning Repository. (2009). *Wine quality* [Data set]. <https://archive.ics.uci.edu/dataset/186/wine+quality>

Van Rossum, G., & Warsaw, B. (2001). *PEP 8: Style guide for Python code*. Python Software Foundation. <https://pep8.org>

Appendix

Jupyter Notebook Code

The complete Jupyter Notebook containing all Python code, cell outputs, visualizations, and statistical results is submitted separately as TeamProject.ipynb and is also available at:

<https://github.com/Denaalshaikhli/Wine-Quality-Prediction>

The notebook follows PEP 8 style guidelines (Van Rossum & Warsaw, 2001) and uses the following Python libraries: pandas, numpy, matplotlib, seaborn, scipy, statsmodels, scikit-learn, and pgmpy. Each section of the notebook contains markdown explanation cells that describe the reasoning behind each step and reference the relevant course module. The notebook is organized in the same section order as this report, making it straightforward to trace any result in the report back to its corresponding code cell.

Module Coverage Map

Table 8

Module Coverage Map Showing How Each Course Module Connects to the Analysis

Module	Topic	Notebook Section
Module 1	Descriptive Statistics, Data Types, Visualization	Sections 2, 3, 4a
Module 2	Probability Distributions, Bayes Theorem	Section 6
Module 3	Sampling Distributions, Central Limit Theorem	Section 4b
Module 4	Inferential Statistics, Confidence Intervals	Sections 4c, 5
Module 5	Bayesian Networks	Section 6
Module 6	Generalized Linear Models, Regression	Section 8
Module 7	Classification, Clustering, Model Evaluation	Sections 7, 9

Note. Each module corresponds to one week of the AAI-500 course.